

EA 5.3 Edge Colouring

Di Giovannantonio Marco

1.1 Modellazione

Variabili e Domini

Il problema dell'Edge Colouring con 3 colori può essere modellato come un CSP (X, D, C) come segue.

$$X = \{x_{ij} \mid (i, j) \in E\}$$
$$D(x_{ij}) = \{1, 2, 3\} \quad \forall (i, j) \in E$$

Vincoli

L'insieme C dei vincoli è così definito:

$$C = C_{tri} \cup C_{sym}$$

L'insieme C_{tri} serve per evitare triangoli monocromatici ed è così definito:

$$C_{tri} = \{c_{ijk} \mid 1 \leq i < j < k \leq n \wedge (i, j) \in E \wedge (i, k) \in E \wedge (j, k) \in E\}$$
$$c_{ijk} = (\{x_{ij}, x_{ik}, x_{jk}\}, R_{ijk})$$
$$R_{ijk} = \{(v_{ij}, v_{ik}, v_{jk}) \in D(x_{ij}) \times D(x_{ik}) \times D(x_{jk}) \mid v_{ij} \neq v_{ik} \vee v_{ik} \neq v_{jk} \vee v_{ij} \neq v_{jk}\}$$

L'insieme C_{sym} garantisce che gli archi (i, j) e (j, i) abbiano lo stesso colore, ovvero garantiscono la simmetria, ed è definito così:

$$C_{sym} = \{c_{ij} \mid (i, j) \in E, i < j\}$$
$$c_{ij} = (\{x_{ij}, x_{ji}\}, R_{sym})$$
$$R_{sym} = \{(v_{ij}, v_{ji}) \in D(x_{ij}) \times D(x_{ji}) \mid v_{ij} = v_{ji}\}$$

1.2 Modellazione Minizinc

Ecco il codice Minizinc che modella il problema:

```
1 % define nc as the numbers of colors
2 int: nc = 3;
3 % define n_nodes as the numbers of nodes
4 int: n_nodes = 3;
5 set of int: nodes = 1..n_nodes;
6 % define color matrix (n_nodes x n_nodes)
```

```

7 array[nodes, nodes] of var 1..nc: col;
8
9 % adj matrix (example)
10 array[nodes, nodes] of bool: adj_matrix = [|
11     false, true, true|
12     true, false, true|
13     true, true, false|
14 |];
15
16 % we have a triangle if, for every i, j, k
17 % adj_matrix[i][j] AND adj_matrix[i][k] AND adj_matrix[j][k]
18 % we choose non-deterministically the nodes and check if
    triangle
19 constraint forall(i, j, k in nodes where i < j /\ j < k) (
20     (adj_matrix[i, j] /\ adj_matrix[i, k] /\ adj_matrix[j, k])
21     ->
22     (col[i, j] != col[i, k] /\ col[i, k] != col[j, k] /\ col[i
23         , j] != col[j, k])
24 );
25
26 % we have by track that C[i][j] = C[j][i] (non-direct graph)
27 constraint forall(i, j in nodes where i < j) (
28     col[i, j] == col[j, i]
29 );
30
31 % helps the solver to be faster
32 % default color for not-triangles arc is 1
33 constraint forall(i, j in nodes) (
34     not adj_matrix[i, j] -> col[i, j] == 1
35 );
36
37 solve satisfy;
38
39 output [
40     "Arc(\(i), \(j)):Color\(col[i,j])\n"
41     | i, j in nodes where i < j /\ adj_matrix[i,j]
42 ];

```